

Cannon Code Breaking

Written by: Luke Verlangieri (lverl23@uw.edu)

This is a physics and coding based problem. Given a cannon, a code, and locations of numbered buckets recreate the code by hitting the buckets in the order the numbers appear in the code. This problem can be solved with simple projectile motion and some good programming practices.

The Story

Its World War 3, the enemy is approaching your position, and you need a way to defend yourself. You find yourself in a 2D plane with only a cannon and information about enemies surrounding artillery bases. The bases are linked together with an elaborate system, such that if the bases are destroyed out of order, a giant explosion will occur, wiping out everyone in your 2D plane (bad). You must stop the enemy by shooting the cannon and hitting the right bases in order, or else humanity is doomed.

To complete this task you must lean on your knowledge of projectile motion as well as your ability to program. The cannon only takes inputs of launch angles and velocities in a list, so make sure you feed the cannon the right angles and velocities to hit the right targets in sequence!

The Programming Interface

Your code will go in the `Cannoneer.py` file found in this folder. Through this file you will interact with a simulation tool developed for this challenge that being the `Cannon_Simulator.py`. Please look into the source code to see what kinds of parameters you can pass to the simulator! Also feel free to edit the source code if you want to make any changes on your local machine. No the only function that you will need to interact with from the cannon simulator class is the `run_sim` function. This function takes in a list of launch angles and velocities as tuples, again, look at the source code for this function. The tuples are expected as (Velocity (m/s), Launch Angle (deg)) in this list.

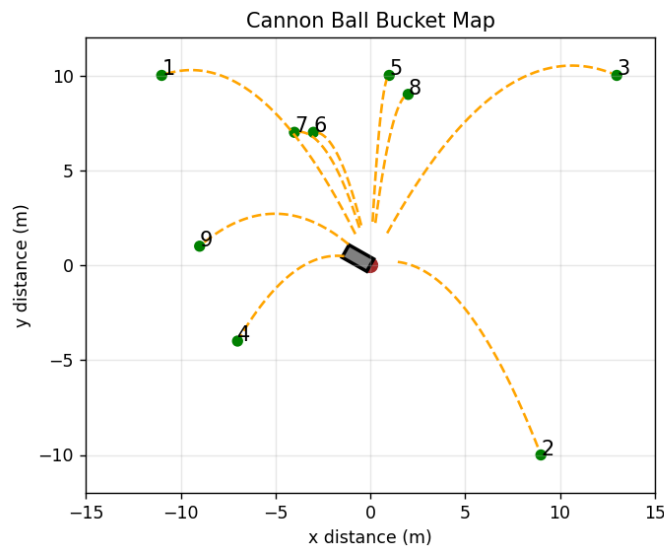


Figure 1: Example of a successfully ran Cannon Simulation

The Challenge

Implement the `cannon_code_breaker` function to shoot buckets of known location in a 2D space in order given a code of non-repeating digits 1-9. Use the `CannonSimulator` class to test your code. Try implementing both a brute force and an analytical solution, and poke around with how changing the simulations parameters impacts your solution.

Deliverable

It is expected that you are able to present your code and speak on the design choices you made and explain how tweaking parameters would change the outcome or your implementation. Please bring your solution to your interview.